

人工智能与机器学习基础

LAB1: 线性回归与正则化

中国科学技术大学

截止时间: 2026 年 10 月 19 日 06:00 (北京时间)

环境创建 (不计分)

沿用 LAB0 已安装的 Miniforge 和 VS Code, 为本实验单独创建名为 `ai3002-lab1` 的 Conda 环境, 不在 LAB0 的 `ai3002` 环境中安装或运行。使用 Python 3.11, CPU 即可, 无需安装 PyTorch 或配置 GPU。本部分不计分, 不提交 OJ 验证码。

Windows

打开 PowerShell, 进入你保存的 LAB1 目录。若课程仓库位于 LAB0 示例位置, 可执行下面的路径命令; 实际位置不同则替换第一行。

```
cd $HOME\ALML_public\LABs\LAB1
conda env create -f environment.yml
conda activate ai3002-lab1
python --version
where.exe python
python -c "import sys; print(sys.executable)"
```

若 PowerShell 无法识别 `conda`, 按 LAB0 的方法打开 Miniforge Prompt, 执行 `conda init powershell`, 关闭并重新打开 PowerShell 后再创建环境。

macOS

打开 Terminal, 进入实际的 LAB1 目录; 下面路径沿用 LAB0 示例。

```
cd "$HOME/ALML_public/LABs/LAB1"
conda env create -f environment.yml
conda activate ai3002-lab1
python --version
which python
python -c "import sys; print(sys.executable)"
```

若无法识别 `conda`, 参照 LAB0 初始化当前 Shell, 并重新打开终端。不要把 Windows 的路径命令复制到 macOS 终端。

选择解释器并检查

在 VS Code 中打开 LAB1 文件夹，Windows 按 `Ctrl+Shift+P`、macOS 按 `Cmd+Shift+P`，执行 `Python: Select Interpreter`，选择 `ai3002-lab1` 对应的 Python 3.11。新建集成终端，确认环境已激活，再在 LAB1 根目录执行：

```
python src/verify_env.py
```

脚本检查新环境名称、Python 版本、NumPy、scikit-learn 和 Matplotlib，输出 `LAB1 environment ready`，并在 `results/` 下生成环境记录和绘图检查文件。环境记录仅用于排错，不占分。若解释器仍指向 `base` 或 `ai3002`，重新选择解释器并激活新环境，不要修改检查脚本。

后续每次实验先执行 `conda activate ai3002-lab1`，所有命令均在 LAB1 根目录运行，无需进入 `src/`。用 `conda info --envs` 查看环境；已创建成功的环境不必重复创建，用完执行 `conda deactivate` 即可。

```
LAB1/  
|-- lab1.pdf  
|-- README.md  
|-- environment.yml  
|-- requirements.txt  
|-- src/  
|   |-- submission.py  
|   |-- run.py  
|   |-- supplied.py  
|   |-- config_protocol.py  
|   |-- benchmark.py  
|   |-- verify_env.py  
|-- data/  
|   |-- dev.csv  
`-- results/
```

0 实验介绍

本实验用线性模型预测设备一次运行的能耗，熟悉数据预处理、模型实现、训练、验证和测试的完整流程。预测发生在运行开始之前，模型将用于训练数据中没有出现过的新设备。

个人完成，CPU 即可，预计 8–10 小时，满分 100 分：OJ 系统评分 50 分（代码 40 分、最终性能 10 分），实验报告评分 50 分。必做内容为第 1、2 部分和第 4 部分的四个问题。环境与提交要求见 `README.md`。补全 `src/submission.py` 中的七个计算函数，调参后填写 `get_config()`，并完成学习率与正则化参数的两轮调整；数据读取、固定划分、实验循环、画图和模型保存由 `src/run.py` 提供。

0.1 数据与文件

`data/dev.csv` 包含 60 台设备的 600 条记录，每台设备 10 条。`supplied.load_development()` 返回按 `row_id` 排序的数组。原始特征 x 的形状为 $(600, 29)$ ，标签 y 的形状为 $(600,)$ 。

字段	含义与用途
<code>load</code> 、 <code>temperature</code> 、 <code>setting</code>	运行前的负载、温度与设定值；可作特征
<code>load_near</code> 、 <code>temperature_milli</code>	相关或不同单位的运行前测量；可作特征
<code>nuisance_00</code> 至 <code>nuisance_23</code>	24 个其他运行前测量；可作特征
<code>row_id</code> 、 <code>device_id</code>	行索引、设备编号；用于匹配和划分，不作特征
<code>post_cycle_meter</code>	运行结束后的读数；只在第 2.3 节指定对照中使用
<code>y</code>	待预测能耗；不能作为输入特征

数据为教学合成数据，无缺失值，不要填补或删除样本。特征顺序由 `supplied.SENSOR_NAMES` 指定。本实验使用固定设备划分，不要求交叉验证。

文件	作用
<code>src/submission.py</code>	补全七个待实现函数，是需要修改的代码文件
<code>src/run.py</code>	提供检查、比较、最终测试与保存流程，不需要修改
<code>src/supplied.py</code> 、 <code>data/results/</code>	提供数据读取和小规模检查样例，不需要修改 运行脚本自动生成结果；不要手工编辑结果

0.2 固定划分与模型约定

脚本使用种子 20260918 打乱排序后的设备编号，前 48 台用于训练，接着 6 台用于验证，最后 6 台用于本地测试；对应 480、60、60 条记录，比例为 8:1:1。同一设备不能跨集合。每次拟合的预处理统计量只能从当前训练部分计算。

先在训练集拟合，用验证集选择配置；配置确定后，合并训练集与验证集重训，在本地测试集评价一次；最后用全部 600 条记录重训并在本地保存最终模型。教师另保留新设备数据用于第 6 节的最终性能计分，不要求学生提供该数据的预测成绩。不得根据本地测试成绩继续选择特征或参数。

本实验允许截距，通过特征标准化和标签中心化实现。设当前训练特征为 $F \in \mathbb{R}^{n \times p}$ ，标签为 $y \in \mathbb{R}^n$ 。其中 n 为训练样本数， p 为特征数， i 为样本下标， j 为特征下标。定义

$$\mu_j = \frac{1}{n} \sum_{i=1}^n F_{ij}, \quad s_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (F_{ij} - \mu_j)^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i.$$

若 $s_j < 10^{-12}$ ，将该列除数置为 1。记标准化矩阵为 Z ，中心化标签为 y_c ，系数为 $w \in \mathbb{R}^p$ ：

$$Z_{ij} = \frac{F_{ij} - \mu_j}{s_j}, \quad (y_c)_i = y_i - \bar{y}, \quad \hat{y}_i = \bar{y} + \sum_{j=1}^p \frac{F_{ij} - \mu_j}{s_j} w_j.$$

对验证或测试输入使用已保存的训练均值与标准差，不重新计算；标签只中心化，不除以标准差。截距不受正则化惩罚。以下计算均使用双精度浮点数。

1 实现线性回归（代码 40 分）

允许使用 NumPy 数组运算、`lstsq`、`solve` 和 `eigvalsh`。这七个函数内不得调用机器学习库的回归器、标准化器、指标函数或自动求导；允许在选做部分另行使用库函数对照。不要求自己实现矩阵分解。

1.1 数据预处理（8 分）

(a) 实现 `make_features(X, basis)`（3 分）。`linear` 原样保留 29 列；`quadratic` 在这 29 列之后，依次追加下列三列，返回 32 列。不要生成其他交叉项，也不要加入常数列。

$$\text{load}^2, \quad \text{load temperature}, \quad \text{temperature}^2.$$

(b) 实现 `fit_preprocessor(F, y)` 和 `transform(F, prep)`（5 分）。前者返回字典 `mu`、`scale`、`y_mean`，数组形状分别为 $(p,)$ 、 $(p,)$ 和标量；后者只应用已保存的统计量，返回形状与输入相同的矩阵。按第 0.2 节处理常数列，标准差分母使用 n 。不得修改输入数组。

1.2 求解与预测（8 分）

实现 `solve_weights(Z, yc, lam)`。参数 `lam` 对应正则化强度 $\lambda \geq 0$ ，返回形状为 $(p,)$ 的系数。当前目标为

$$J_\lambda(w) = \frac{1}{2n} \|Zw - y_c\|_2^2 + \frac{\lambda}{2} \|w\|_2^2.$$

当 $\lambda = 0$ ，使用 `np.linalg.lstsq(..., rcond=None)` 求普通最小二乘解（4 分）。当 $\lambda > 0$ ，利用下式调用 `np.linalg.solve`，其中 I_p 为 p 阶单位矩阵（4 分）：

$$\left(\frac{Z^\top Z}{n} + \lambda I_p \right) w = \frac{Z^\top y_c}{n}.$$

预测与截距还原已在 `run.py` 中提供。小样例含重复列，普通最小二乘的系数不一定唯一，应检查预测是否一致，不要求自行求逆。

1.3 损失与梯度（8 分）

实现 `loss_gradient(Z, yc, w, lam)`。返回目标函数标量（3 分）及其关于系数的梯度向量（5 分）。用上一节的目标函数自行写出梯度并实现；报告中列出使用的梯度公式即可，无需重复书面作业的推导。

脚本在固定小样例上以步长 $h = 10^{-6}$ 进行中心差分核验，自动记录最大误差。有限差分用于核验，不得作为训练所用的梯度计算方式。

1.4 梯度下降训练（8 分）

实现 `train_gd`。从零系数开始，每次使用全部训练数据，调用自己实现的 `loss_gradient`，按下式更新，其中 t 为更新次数， η 为传入的学习率：

$$w^{(t+1)} = w^{(t)} - \eta \nabla J_\lambda(w^{(t)}).$$

记录初始状态及每次更新后的迭代次数、目标函数值、梯度各分量绝对值的最大值。后者不超过 10^{-7} 时停止，更新次数不得超过传入的 `max_iter`；函数默认上限为 20000 次，供第 1 部分精度检查使用，实际实验须服从各阶段传入的预算。为处理过大学学习率，若目标或梯度出现非有限值，或目标超过初始目标与 1 中较大者的 10^6 倍，记录该状态并停止。返回最终系数和三列历史数组（更新与停止 5 分，记录 3 分），不得隐藏发散记录。

首先在脚本提供的小样例上用 $\lambda = 0.1$ 检查梯度下降实现，检查阶段自动使用以下学习率； $\lambda_{\max}$ 表示对称矩阵的最大特征值。实际设备数据上的学习率调整是第 2.1 节的必做计分任务：

$$\eta = \frac{1}{\lambda_{\max}(Z^\top Z/n) + 0.1}.$$

执行检查命令，阅读 `checks.csv`，查看 `loss_curve.png`，核对梯度下降与直接求解的最大预测差不超过 10^{-5} 。其他检查的容差写在结果文件中。若失败，应定位原因并修正，不要通过修改容差掩盖错误。

```
python src/run.py --stage check --out results
```

1.5 性能评估（8 分）

实现 `evaluate(y, pred)`。对形状为 $(m,)$ 的真实值与预测值返回 `mse`、`rmse`、`mae` 三项标量。其中 m 是当前评价样本数， $\hat{y}_i$ 是预测值：

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}, \quad \text{MAE} = \frac{1}{m} \sum_{i=1}^m |\hat{y}_i - y_i|.$$

评分为均方误差 3 分、均方根误差 2 分、平均绝对误差 3 分。评价误差不包含正则化项。脚本使用真实值 (1, 2, 3, 4) 和预测值 (1, 2, 3, 8) 检查这三个指标。

2 参数调整与性能比较（实验结果 30 分）

本部分将学习率调整计 10 分、正则化参数调整计 12 分、评估对照计 4 分、测试保存计 4 分。运行脚本提供实验工具，你需要依据第一轮结果选择新参数，记录“观察到什么、为什么这样调整、调整后如何”，不能只在报告中粘贴默认脚本输出。

2.1 调整学习率 (10 分)

固定原设备训练集的 480 条记录、二次特征及 $\lambda = 0.1$, 本节观察收敛与发散, 所有试验从零系数开始, 最多更新 2000 次; 第 2.2 节起的模型选择与最终训练统一使用 200 次更新预算。令

$$L = \lambda_{\max}(Z^T Z/n) + 0.1, \quad \eta = \frac{c}{L}.$$

其中 L 由当前训练数据计算, c 是学习率倍数。先运行以下命令, 比较 $c = 0.01$ 、1、2.1 三种情形:

```
python src/run.py --stage tune-lr --data data --out results
```

(a) 初步比较 (4 分)。阅读 `lr_coarse.csv` 和 `lr_curves.png`, 比较目标函数下降、到达指定精度所需的迭代次数及发散情况。三种试验必须使用相同数据、初始化、正则化参数和迭代预算。第 1.4 节的发散停止规则用于保留失败证据, 发散本身不扣分。

(b) 依据结果再调整一次 (4 分)。自行选择一个新的倍数 $c_{\text{new}} \in (0, 2)$, 不能与已试值相同。先在报告中写明选择理由, 再运行带 `--lr-factor` 的命令, 将其与第一轮结果比较。例如下面仅展示命令格式, 不指定必须采用的值:

```
python src/run.py --stage tune-lr --lr-factor 0.5 --out results
```

(c) 比较效率并记录 (2 分)。四组最终记录写入 `lr_tuning.csv`, 完整历史写入 `lr_history.csv`, 选择结果写入 `lr_config.json`。直接解给出同一训练目标的最优值 J_* , 脚本比较各次迭代首次满足下式所需的次数:

$$|J_\lambda(w^{(t)}) - J_*| \leq 10^{-6} \max(1, |J_*|).$$

在未发散且达到上述精度的候选中, 选择所需次数最少的学习率; 次数相同则取较小的倍数。该精度只是统一的比较标准, 不替代梯度停止条件。报告表中注明未达到精度或发散的试验, 不能将较早发散误判为收敛更快。新值没有优于原值, 只要选择与分析合理, 仍可得满分。本节效率最优倍数不直接等于最终提交倍数; 第 2.2 节按验证误差联合选择最终学习率与正则化参数。不得通过测试集选学习率。

2.2 调整正则化参数 (12 分)

固定第 0.2 节的设备划分。先完成第 2.1 节, 再用梯度下降联合比较特征方案、正则化参数与学习率。每个候选从零初始化, 最多更新 200 次; 梯度各分量绝对值的最大值不超过 10^{-7} 时可提前停止, 发散保护沿用第 1.4 节。用停止时的参数评价, 不允许用直接解替代或修正梯度下降结果。

对当前候选的训练特征矩阵 Z 、训练样本数 n 和正则化参数 λ , 脚本重新计算

$$L = \lambda_{\max}(Z^T Z/n) + \lambda, \quad \eta = \frac{c}{L}.$$

其中 c 从第 2.1 节试过的三个稳定范围内倍数中取值: 0.01、1 和你补选的 c_{new} ; 2.1 仅用于发散观察, 不进入模型选择。更换特征、正则化参数或重训数据后都重新计算 L , 不能沿用上一阶段的实际学习率。若 $L = 0$, 脚本取 $\eta = c$, 在初始梯度为零时停止。

(a) **第一轮粗比较 (4 分)**。运行以下命令，对两种特征、三个学习率倍数与以下三个正则化参数的组合进行训练，共 18 个梯度下降候选：

$$\lambda \in \{0, 0.1, 1\}.$$

```
python src/run.py --stage compare --data data --out results
```

脚本自动完成组合训练。`coarse_joint_comparison.csv` 保留 18 行完整结果；`coarse_comparison.csv` 对每个“特征与正则化参数”组合保留验证误差最低的学习率，再加训练标签均值对照，共 7 行。均值对照不参与最终配置选择。查看训练与验证 RMSE、系数范数和迭代状态，决定下一轮探索方向。不能根据本地测试成绩决定方向。

(b) **自行补选两个值 (4 分)**。根据第一轮验证结果，选两个互不相同的新正则化参数，不得重复已有值。报告选择方向与理由，再通过 `--extra-lambdas` 传入；脚本自动在两种特征和三个学习率倍数下运行，无需增加手工试验轮数。下面仅为命令格式示例：

```
python src/run.py --stage compare --extra-lambdas 0.02 0.05 --out results
```

(c) **冻结最终配置 (4 分)**。第二轮保留原有组合，`joint_comparison.csv` 共 30 行；按每个特征与正则化参数组合保留最佳学习率，再加均值对照，得到 11 行 `comparison.csv` 和 660 行 `validation_predictions.csv`。报告两轮汇总表、最终三项配置，并从完整表中引用同一特征、同一正则化参数下不同学习率的验证误差，说明有限训练预算的影响。无需重复粘贴全部 30 行。

`regularization_curves.png` 固定 $c = 1$ ，显示正则化参数变化时的训练与验证 RMSE、系数范数；不能把同时改变学习率的差异全部归因于正则化。预算内的参数未必达到目标最优解，因此不要求误差或范数严格单调。

脚本在 30 个梯度下降候选中按未舍入验证 RMSE 选择最终配置；精确相同时依次优先线性特征、较大的正则化参数、较小的学习率倍数。结果写入 `config.json`，不要手工改写。完成学习率试验后再开始本节，模型比较开始后不得返回修改学习率候选；不得看过测试成绩后重新选择。新增值不必优于第一轮结果，不要求继续无上限搜索。

2.3 两项评估对照 (4 分)

`compare` 命令同时生成三行 `audit.csv`。三种条件都固定二次特征、正则化参数 $\lambda = 0.1$ 、学习率倍数 $c = 1$ 和 200 次梯度下降更新预算，不在对照中另行调参。

条件	划分与特征
group	原设备划分，480 条训练、60 条验证，32 个二次特征
random	仅将上述 540 条训练及验证记录重新按行打乱为 480、60 条；特征同 group
post	划分同 group，额外加入运行结束后的 <code>post_cycle_meter</code>

三种条件均不使用已保留的 60 条本地测试记录。`random` 使用相同随机种子，其训练和验证中可能出现相同设备。`post` 是指定的错误信息使用示例，不能成为最终模型的候选。

你需要做：报告三个 RMSE，以及 random 相对 group、post 相对 group 的 RMSE 差；报告脚本统计的训练与验证共享设备数。回答第 4 部分对应问题。不要求任何对照一定取得更低误差，也不要求显著性检验。

计分：同模型与同参数的三行对照完整 2 分，两项误差差值正确 1 分，共享设备数正确 1 分。解释的分数另计入报告，不重复扣分。

2.4 测试与保存（4 分）

完成两项调参及对照分析后，把所选配置写入 `src/submission.py` 的 `get_config()`。此接口已提供，只需填写三个值，不增加计算函数或分值：

```
def get_config():
    return {
        "basis": None,
        "lambda": None,
        "lr_factor": None,
    }
```

将 `basis`、`lambda`、`lr_factor` 分别填写为 `config.json` 中的 `basis`、`lam`、`lr_factor`。三项均来自联合验证选择，不要填写第 2.1 节的 `chosen_factor` 或实际学习率。应原样复制所选数值，不做四舍五入。

`basis` 只允许 `linear` 或 `quadratic`；训练标签均值仅作为对照，不允许提交为最终模型。正则化参数须为有限非负数，学习率倍数须为有限数且在开区间 $(0, 2)$ 内。接口必须返回上述三个字段，不得读取本地配置文件、结果目录或执行参数搜索；提交前不能保留 `None`。

最终训练调用 `train_gd()`，使用所选特征、正则化参数和学习率倍数；本地测试重训与 OJ 最终训练均从零开始、最多更新 200 次、使用相同梯度停止阈值。学习率和正则化参数共同影响预算内得到的系数及最终性能。不得增加预算、热启动、替换为直接解或伪造训练历史。直接求解函数仅用于第 1 部分核验和第 2.1 节的精度参照。

冻结代码中的配置后，运行：

```
python src/run.py --stage final --data data --out results
```

脚本检查已经完成学习率补选与正则化参数补选，核对 `get_config()` 与两份选参记录一致；不一致则在测试前报错。通过后，脚本读取该接口，保存 `submitted_config.json`，再按第 0.2 节重训、测试、用全部数据重训，生成 `test_metrics.csv`、60 行 `test_predictions.csv` 和 `final_model.npz`。报告三个本地测试指标，确认 `run.predict_saved(model_path, raw_X)` 可以加载模型。生成的结果和模型文件仅留存本地，不上传。计分：配置冻结与正确重训 2 分，模型可加载且预测一致 2 分。

已存在测试结果时，脚本阻止再次调参或重复测试。正常调试应在测试前完成；若发现导致测试无效的程序错误，可修正后如实记录原因及重跑过程，不得利用已见测试成绩修改参数。

3 选做探索（不计分）

可以另与 scikit-learn 的 Ridge 对照，或研究特征尺度对梯度下降的影响。选做结果单独标注，不替换规定的两轮调参，不依据本地测试结果做探索。不要求实现 Lasso、分类器、重复抽样稳定性分析或偏差与方差的独立模拟。

4 报告问题（16 分）

按题号回答以下四个问题，每题 4 分。每题建议一段文字，引用自己的结果；可以附一条必要公式，不需要写长篇理论综述。

(1) 预处理。为什么验证集必须使用训练集的均值和标准差？小样例中的常数列如何处理？结合你保存的检查结果说明实现是否符合这一要求。

(2) 训练。写出梯度公式，结合有限差分检查及四条学习率曲线，解释小步长、大步长与所选新步长的表现。为什么不能只看“损失下降”或最终迭代次数判断训练正确？区分学习率选择与正则化参数选择的作用。

(3) 正则化。结合两轮结果，解释你为何选择两个新参数。训练误差、验证误差和系数范数如何变化？选中配置是否改变？说明为何系数更小或训练更充分不意味着验证误差必然更低，不把一次划分的结果当成普遍规律。

(4) 评估。结合第 2.3 节的差值与共享设备数，解释按行划分和加入事后字段分别改变了什么。哪种条件与“运行前预测新设备”一致？说明为什么设备隔离不能修复事后字段问题，以及为什么单次对照不能精确量化所有场景下的乐观程度。

5 流程记录与反馈（4 分）

报告正文建议 4-6 页：简述流程，列出四组学习率试验、两轮参数比较、三个测试指标和三行对照；放入 `lr_curves.png` 与 `regularization_curves.png`，再回答四个问题。初始检查曲线可放附录，第一轮结果可在最终 11 行表中标注，无需重复贴表。必须保留调参前的观察与选择理由，不求长篇推导。

流程与运行记录 2 分：写清环境、命令、配置选择和重训顺序。AI 使用或自检记录 1 分：写明使用工具及用途，选一个关键建议或假设说明如何通过运行独立核验；未使用 AI 则记录等价自检。反馈 1 分：如实填写完成实验的大致用时，可另提课程建议。

允许使用 AI 辅助，但必须理解并能解释提交的代码、图表和结果。总分 100 分。OJ 系统负责代码 40 分与最终性能 10 分的评分，共 50 分；Blackboard 实验报告占其余 50 分，包括调参与实验结果 30 分、四个报告问题 16 分、流程记录与反馈 4 分。环境创建不计分；最终性能按第 6 节与固定基准比较。实验报告以 PDF 格式上传至 Blackboard 系统 (bb.ustc.edu.cn)，`submission.py` 单独上传至 OJ (oj.temaurinum.moe)；不接收其他文件。图表、参数选择和运行记录写入报告，不另交结果目录、模型文件或压缩包。报告须记录补选学习率、两个补选正则化参数、最终配置和完整运行命令，以便复现。报告须与代码中的配置一致。OJ 的代码 40 分独立测试七个计算函数，不因最终特征方案而免测。

6 最终性能评测（10 分）

本项 10 分由 OJ 系统评分。OJ 仅从 `submission.py` 读取 `get_config()`，在全部开发数据上调用提交的特征、预处理和 `train_gd()`，按第 2.2 节的 200 次预算与停止条件训练模型，学习率与正则化参数均来自该配置。不读取报告、配置文件或已保存模型。OJ 以 RMSE 将所得模型与固定的 scikit-learn Ridge 基准（二次特征，正则化参数 $\lambda = 0.1$ ）比较。RMSE 不高于基准得 10 分；否则，记基准与学生模型的 RMSE 分别为 E_b 和 E_s ，性能得分 P 按以下比例计算：

$$P = 10 \frac{E_b}{E_s}.$$

`final_model.npz` 仅在本地保存，无需上传。`performance_preview.csv` 仅为本地预估，不作为正式得分，也不能据此继续调参。